

GenAI Assistant with RAG – Assignment Reference Guide

1. Objective

Project Goal

The goal of this project is to build a **GenAI-powered Chat Assistant** that can answer user questions based on information stored in a custom document knowledge base.

Expected Final Outcome

At the end of the project, you should have:

- A Python backend API
 - A document knowledge base
 - Embedding generation system
 - Vector storage
 - Similarity search mechanism
 - LLM integration
 - Basic HTML chat interface
 - Session-based conversation history
 - Grounded AI responses using retrieved documents
-

2. Technical Requirements

Programming Language

- Python 3.10+

Backend

Choose one:

- FastAPI (Recommended)
- Flask

Frontend

- HTML
- CSS
- JavaScript

LLM Providers

Choose one:

- OpenAI
- Gemini
- Claude
- Mistral

Embedding Models

Examples:

- OpenAI Embeddings
- Gemini Embeddings
- Sentence Transformers

Storage

Choose one:

- In-Memory Storage
- SQLite
- Simple JSON-based storage

Python Libraries

Recommended:

fastapi
uvicorn
python-dotenv
numpy
scikit-learn
openai
sentence-transformers

Development Tools

- VS Code
- Git
- GitHub
- Postman (optional)

Environment Requirements

Create a `.env` file:

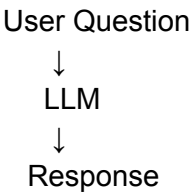
```
LLM_API_KEY=your_api_key  
EMBEDDING_API_KEY=your_api_key
```

3. Step-by-Step Implementation Guide

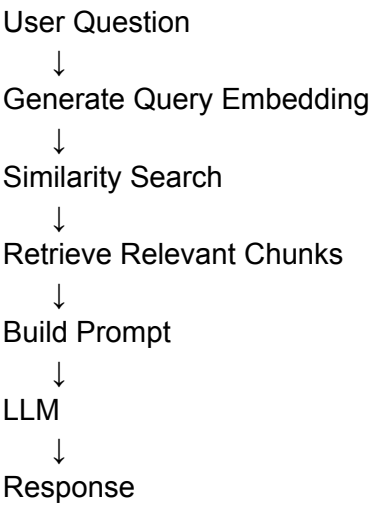
Step 1: Understand the RAG Workflow

Before writing code, understand the complete flow.

Normal Chatbot



RAG Chatbot



Why This Is Needed

Without retrieval:

- AI guesses answers.

With retrieval:

- AI answers using actual documents.

Step 2: Create the Knowledge Base

Create:

docs.json

Example:

```
[
  {
    "title": "Reset Password",
    "content": "Users can reset their password from Settings > Security."
  }
]
```

```
},
{
  "title": "Account Deletion",
  "content": "Users can delete their account from Account Settings."
}
]
```

Why This Is Needed

This acts as the company's internal knowledge base.

The chatbot will answer questions only from this information.

Step 3: Implement Document Chunking

Large documents should not be stored as one huge block.

Example

Document:

1000-word article

Split into:

- Chunk 1
- Chunk 2
- Chunk 3
- Chunk 4

Recommended chunk size:

300–500 tokens

Why Chunking Is Important

Benefits:

- Better retrieval accuracy
- Faster search
- More relevant context

Expected Output

```
[
  "chunk one",
  "chunk two",
  "chunk three"
]
```

Step 4: Generate Embeddings

What Is an Embedding?

An embedding is a numerical representation of text.

Example:

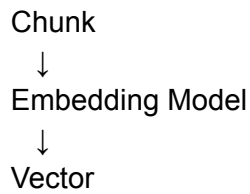
"Reset password"

may become:

[0.23,-0.56,0.88, ...]

Process

For every chunk:



Store:

```
{  
  "text": chunk,  
  "embedding": [...]  
}
```

Why This Is Needed

Computers cannot compare text directly.

Embeddings allow mathematical comparison between texts.

Step 5: Store Embeddings

Store:

```
[  
  {  
    "chunk": "...",  
    "embedding": [...]  
  }  
]
```

Possible storage options:

Option 1 (Simple)

In-memory list

Option 2

SQLite

Option 3

JSON file

Why This Is Needed

Embeddings must be stored so retrieval can happen later without regenerating vectors.

Step 6: Generate Query Embedding

When a user asks:

How do I reset my password?

Generate embedding:

Question



Embedding Model



Vector

Why

Now both:

- Documents
- User question

exist in the same vector space.

Step 7: Implement Similarity Search

Compare:

Question Embedding

with

Document Embeddings

Common Methods

Cosine Similarity

Most commonly used.

Measures how close two vectors are.

Score range:

0 → Not Similar

1 → Very Similar

Retrieval Process

Query Vector



Compare with all vectors



Rank results



Select Top 3

Why

This identifies the most relevant information.

Step 8: Apply Similarity Threshold

Example:

threshold=0.75

Scenario 1

Similarity:

0.89

Result:

Use retrieved chunk

Scenario 2

Similarity:

0.20

Result:

Insufficient information

Why

Prevents irrelevant answers.

Reduces hallucinations.

Step 9: Build the RAG Prompt

Combine:

Retrieved Context

Password can be reset from Settings > Security.

Conversation History

User: Hello

Assistant: Hi

Current Question

How do I reset my password?

Prompt Structure

You are a helpful assistant.

Use only the provided context.

Context:

...

History:

...

Question:

...

Answer:

Why

This ensures answers are grounded in retrieved data.

Step 10: Integrate the LLM

Send the constructed prompt to:

- OpenAI
- Gemini
- Claude
- Mistral

Recommended settings:

temperature=0.2

Why Low Temperature?

Produces:

- More consistent answers
 - Less hallucination
 - More factual responses
-

Step 11: Handle Failures

Your system should gracefully handle:

Invalid API Key

Return:

```
{  
  "error": "Invalid API key"  
}
```

Timeout

Return:

```
{  
  "error": "Request timeout"  
}
```

Rate Limit

Return:

```
{  
  "error": "Rate limit exceeded"  
}
```

Why

Production systems must never crash unexpectedly.

Step 12: Store Conversation History

Maintain:

Last 3–5 message pairs

Example:

```
{  
  "sessionId":"abc123",  
  "history": [...]  
}
```

Why

Allows follow-up questions.

Example:

User: How do I reset my password?

Assistant: ...

User: Where is that option?

The assistant can understand "that option".

Step 13: Create Chat API

Endpoint

POST /api/chat

Request

```
{  
  "sessionId":"abc123",  
  "message":"How can I reset my password?"  
}
```

Response

```
{  
  "reply":"Users can reset their password from Settings > Security.",  
  "tokensUsed":120,  
  "retrievedChunks":3  
}
```

Validation

Check:

- Empty message
 - Missing sessionId
 - Invalid JSON
-

Step 14: Build Frontend UI

Required Components:

Input Box

User enters message.

Send Button

Submits request.

Message Area

Displays chat conversation.

Loading Indicator

Shows processing state.

Session Storage

Store:

```
localStorage.setItem("sessionId",id);
```

Why

Provides a usable chat experience.

Step 15: Test End-to-End Flow

Test:

Valid Question

How do I reset my password?

Expected:

Correct retrieved answer

Unknown Question

What is your refund policy?

Expected:

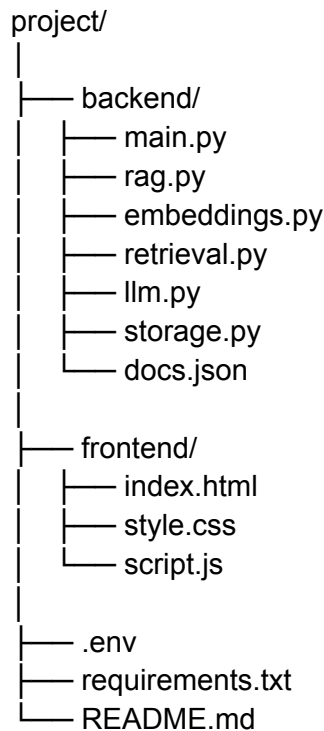
I do not have enough information to answer that.

Why

Ensures retrieval is working correctly.

4. Project Structure

Example Structure:



Important Files

docs.json

Knowledge base documents.

[embeddings.py](#)

Generates embeddings.

[retrieval.py](#)

Performs similarity search.

[rag.py](#)

Coordinates retrieval and prompt creation.

[llm.py](#)

Handles LLM API communication.

[main.py](#)

API entry point.

script.js

Frontend interaction logic.

5. Example Code Snippets

Example: Load Documents

```
import json

with open("docs.json") as f:
    documents = json.load(f)
```

Example: Generate Embedding

```
embedding = embedding_model.embed(text)
```

Purpose:

Convert text into vector representation.

Example: Similarity Search

```
from sklearn.metrics.pairwise import cosine_similarity

score = cosine_similarity(
    [query_vector],
    [document_vector]
)
```

Purpose:

Measure semantic similarity.

Example: Sort Top Results

```
results.sort()
```

```
key=lambda x:x["score"],
reverse=True
)
```

Purpose:

Retrieve most relevant chunks first.

Example: Threshold Check

```
if best_score < 0.75:
    return "Insufficient information."
```

Purpose:

Avoid hallucinated responses.

Example: Prompt Construction

```
prompt=f"""
Context:
{retrieved_context}

History:
{conversation_history}

Question:
{user_question}
"""
```

Purpose:

Provide grounding information to the LLM.

Example: API Route

```
@app.post("/api/chat")
def chat(request):
    return {"reply": response}
```

Purpose:

Expose chatbot functionality.

Example: Frontend API Call

```
fetch("/api/chat", {  
  method:"POST",  
  body:JSON.stringify(data)  
});
```

Purpose:

Send user messages to backend.

Additional Submission Requirements

Students must submit:

1. GitHub Repository Link
2. Hosted Live Application Link (deployed version)
3. [README.md](#) containing:
 - Architecture Diagram
 - RAG Workflow Explanation
 - Embedding Strategy
 - Similarity Search Logic
 - Prompt Design Reasoning
 - Setup Instructions
 - Screenshots
4. Demo Video (Recommended)